

Relatório MIPS Pipeline

Nomes: Ana Luiza Boeng, Gabriele Bueno Santos, Marcello Eduardo Pereira Jorge e Mona Aya Kanso.

Estágio IF (Instruction Fetch)

Visto que o conjunto de instruções implementado não possui instruções de desvio (branch ou jump), não houve necessidade da utilização de um multiplexador (MUX) para seleção do próximo valor do Program Counter (PC). Dessa forma, o PC apenas incrementa seu valor a cada ciclo de clock, realizando a busca sequencial das instruções armazenadas na memória.

O estágio IF foi implementado utilizando os seguintes componentes:

- Program Counter (PC);
- Incrementador;
- Memória ROM de instruções;
- Registrador de pipeline IF/ID.

Para verificar o funcionamento do estágio, foi desenvolvido um testbench isolado. O objetivo do teste foi validar o incremento correto do PC, o acesso à memória ROM e o armazenamento das instruções no registrador IF/ID.

Foram inseridas instruções fictícias manualmente na memória ROM para facilitar a visualização e a validação dos resultados. A simulação foi realizada utilizando o simulador ISim da ferramenta Xilinx ISE.

Os resultados obtidos demonstraram o funcionamento correto do estágio IF. Observou-se que o PC foi incrementado sequencialmente a cada ciclo de clock, acessando corretamente as posições da memória ROM. Além disso, verificou-se que as instruções lidas foram armazenadas pelo registrador IF/ID e disponibilizadas para o próximo estágio do pipeline com atraso de um ciclo de clock, comportamento esperado para uma arquitetura pipeline.

A seguir são apresentados a memória ROM utilizada no teste, o gráfico de simulação e o relatório gerado pelo simulador.

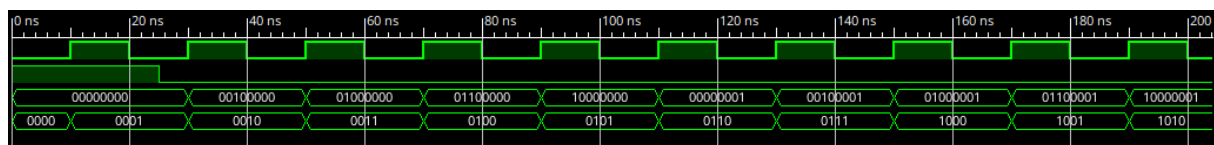
Memória ROM utilizada no teste

```

0 => "00000000",
1 => "00100000",
2 => "01000000",
3 => "01100000",
4 => "10000000",
5 => "00000001",
6 => "00100001",
7 => "01000001",
8 => "01100001",
9 => "10000001",
10 => "00000010",
11 => "00100010",
12 => "01000010",
13 => "01100010",
14 => "10000010",
15 => "00000011",

```

Gráfico de simulação



Relatório do simulador

Finished circuit initialization process.

at 31 ns: Note: PC = 0010 (2) | IF_ID = 00100000 (32) (/tb_if_stage/).
at 51 ns: Note: PC = 0011 (3) | IF_ID = 01000000 (64) (/tb_if_stage/).
at 71 ns: Note: PC = 0100 (4) | IF_ID = 01100000 (96) (/tb_if_stage/).
at 91 ns: Note: PC = 0101 (5) | IF_ID = 10000000 (128) (/tb_if_stage/).
at 111 ns: Note: PC = 0110 (6) | IF_ID = 00000001 (1) (/tb_if_stage/).
at 131 ns: Note: PC = 0111 (7) | IF_ID = 00100001 (33) (/tb_if_stage/).
at 151 ns: Note: PC = 1000 (8) | IF_ID = 01000001 (65) (/tb_if_stage/).
at 171 ns: Note: PC = 1001 (9) | IF_ID = 01100001 (97) (/tb_if_stage/).
at 191 ns: Note: PC = 1010 (10) | IF_ID = 10000001 (129) (/tb_if_stage/).
at 211 ns: Note: PC = 1011 (11) | IF_ID = 00000010 (2) (/tb_if_stage/).
at 231 ns: Note: PC = 1100 (12) | IF_ID = 00100010 (34) (/tb_if_stage/).
at 251 ns: Note: PC = 1101 (13) | IF_ID = 01000010 (66) (/tb_if_stage/).
at 271 ns: Note: PC = 1110 (14) | IF_ID = 01100010 (98) (/tb_if_stage/).
at 291 ns: Note: PC = 1111 (15) | IF_ID = 10000010 (130) (/tb_if_stage/).
at 311 ns: Note: PC = 0000 (0) | IF_ID = 00000011 (3) (/tb_if_stage/).
at 331 ns: Note: PC = 0001 (1) | IF_ID = 00000000 (0) (/tb_if_stage/).

Instruções:

Instruções do tipo R																	
NOME	OPCODE			RS			RT			RD			FUNC		REGISTRADORES:	DESCRIÇÃO	
NOP	0	0	0	0	0	0	0	0	0	0	0	0	0	0	R0	zero	
ADD	0	0	1									0	0	0	1	R1	resultado do add
SUB	0	1	0									0	0	1	0	R2	resultado do sub
AND	0	1	1									0	0	1	1	R3	resultado do and
Instruções tipo I																	
NOME	OPCODE			RS			RT			IMEDIATO							
LOAD	1	0	0												R4		
STORE	1	0	1												R5		
ADDI	1	1	0												sp	stack pointer (aponta pro último endereço)	

Memória de Instruções e Memória de Dados

Nesta etapa do projeto foram desenvolvidos os módulos de Memória de Instruções e Memória de Dados da arquitetura pipeline.

A Memória de Instruções foi implementada como uma memória ROM responsável por armazenar as instruções do programa, fornecendo a instrução correspondente ao endereço recebido. Já a Memória de Dados foi desenvolvida para permitir operações de leitura e escrita durante a execução das instruções, utilizando sinais de controle para habilitar cada operação.

...